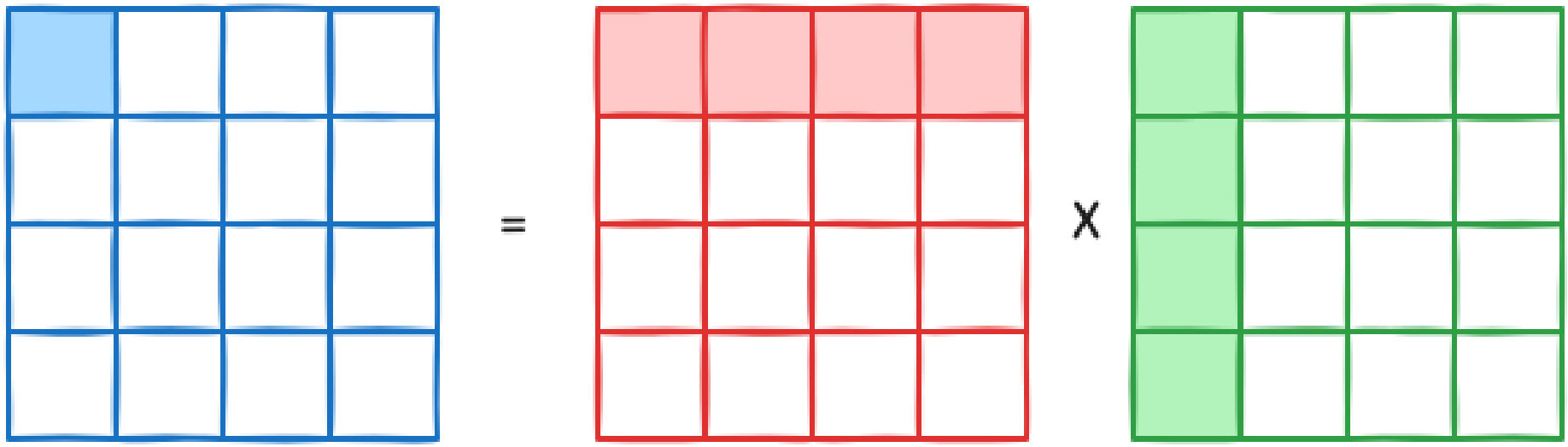
Table of Contents

1. Why GPUs? What's wrong with CPUs?

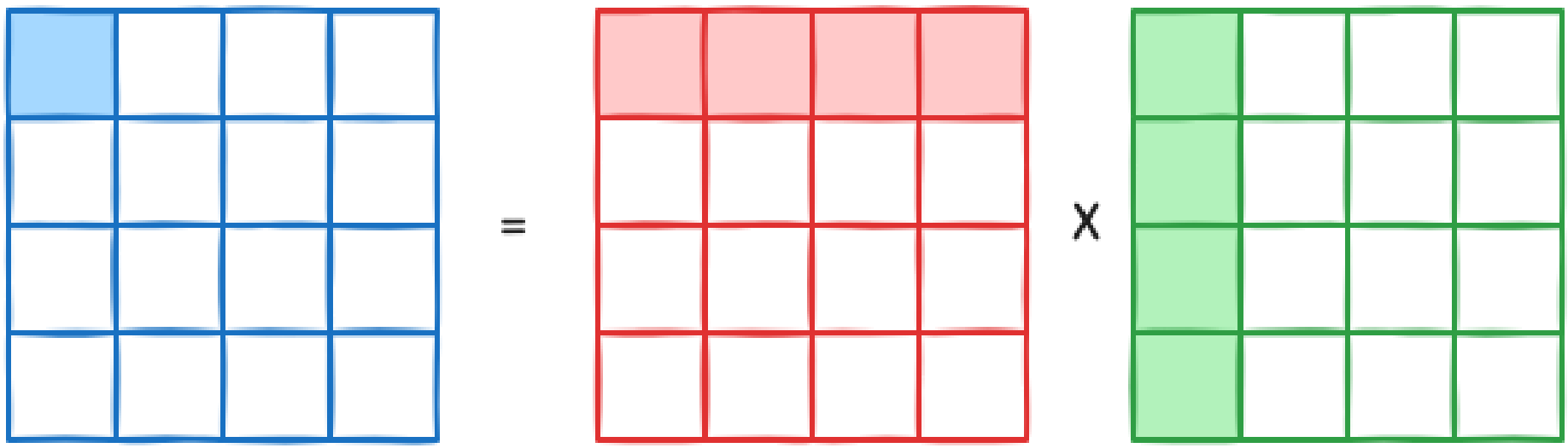
Computing a Matrix Multiplication

Let A and B be two 4×4 matrices. The matrix multiple $C = AB$ results in another 4×4 matrix.



Computing a Matrix Multiplication

Let A and B be two 4×4 matrices. The matrix multiple $C = AB$ results in another 4×4 matrix.



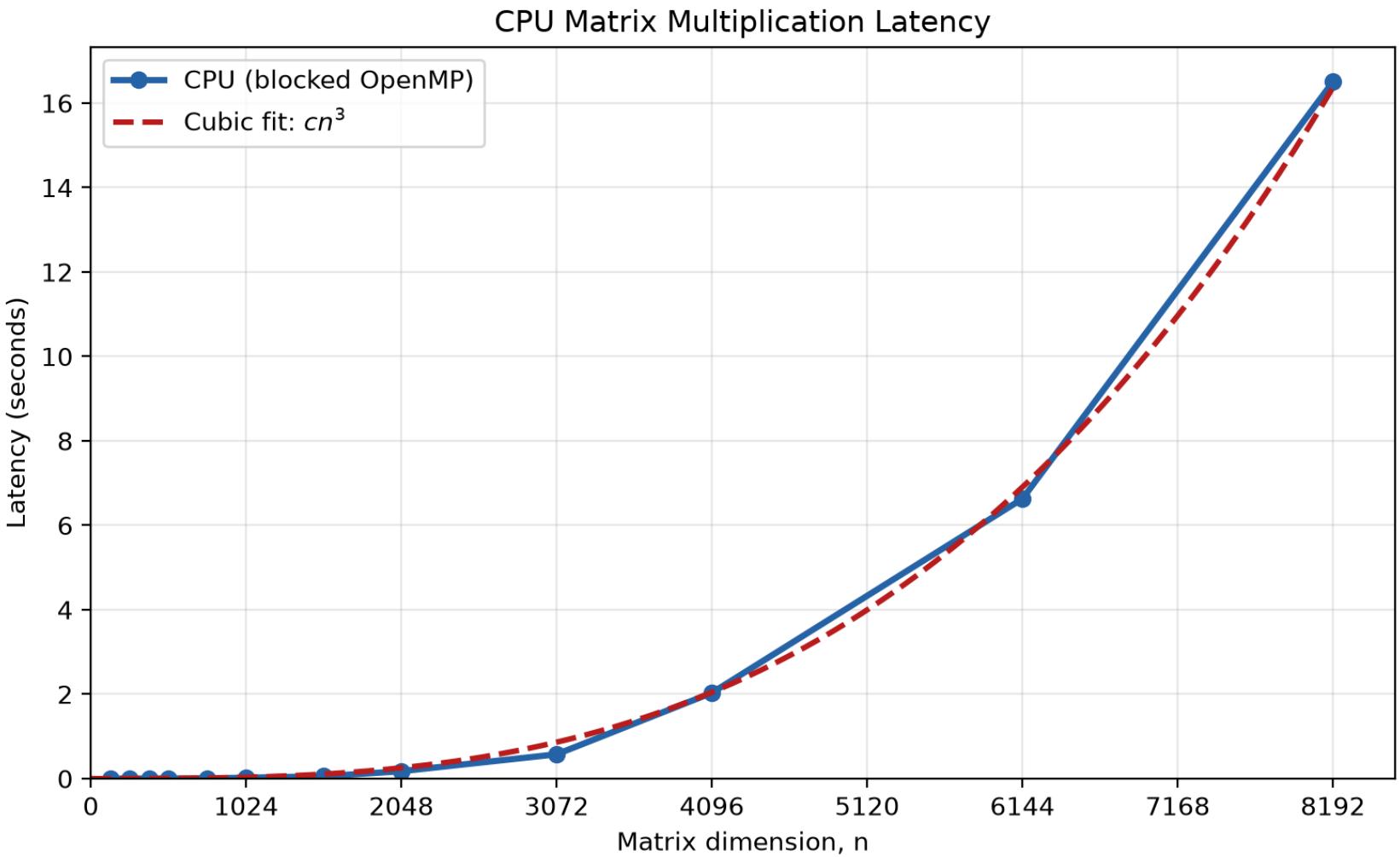
Poor scaling

The operations are very simple, but the number of operations grow on the order of $\mathcal{O}(n^3)$, where n is the dimensions of the matrices.

Runtime of a matrix multiplication on a CPU

```
for (std::size_t y = 0; y < n; ++y) {
  for (std::size_t x = 0; x < n; ++x) {
    double sum = 0.0;
    for (std::size_t k = 0; k < n; ++k) {
      sum += a[y*n+k] * b[k*n+x];
    }
    c[y*n+x] = sum;
  }
}
```

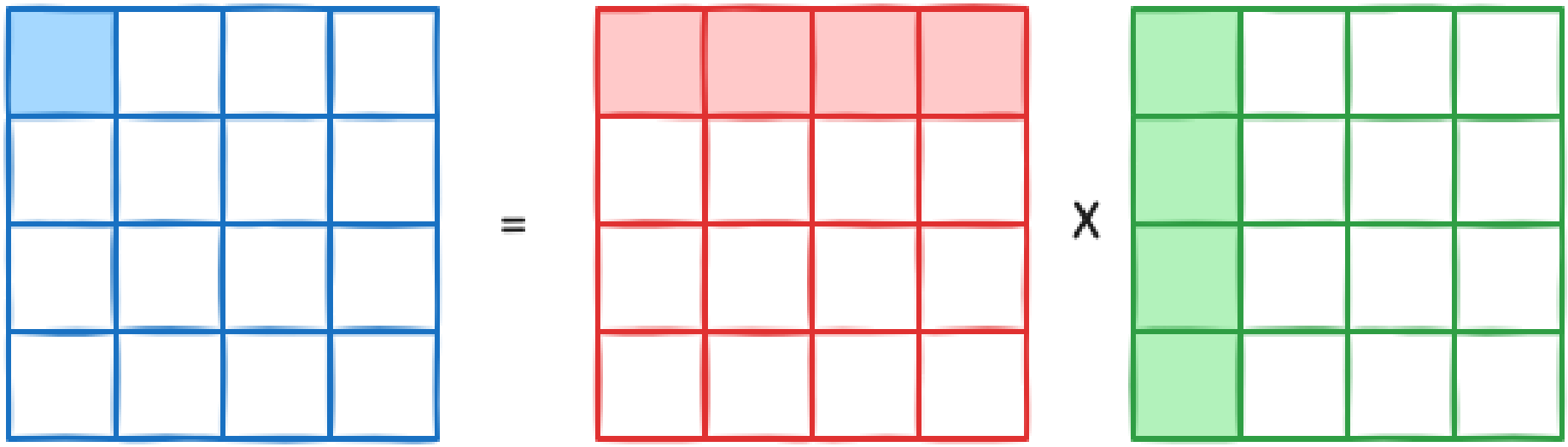
Runtime of a matrix multiplication on a CPU



Parallelism

GPU's came about due to the increase in computational power needed to render 3D games in real time. This requires *millions* of FLOP's per second. CPU's could not keep up. Rendering graphics has a nice property that we can take advantage of: **parallelism**.
Each pixel on the screen doesn't really care any of the other pixels. Furthermore, the computation that is required for each pixel isn't that complex either. The difficulty in rendering graphics is the volume of computations.

Matrix Multiplication is Parallel

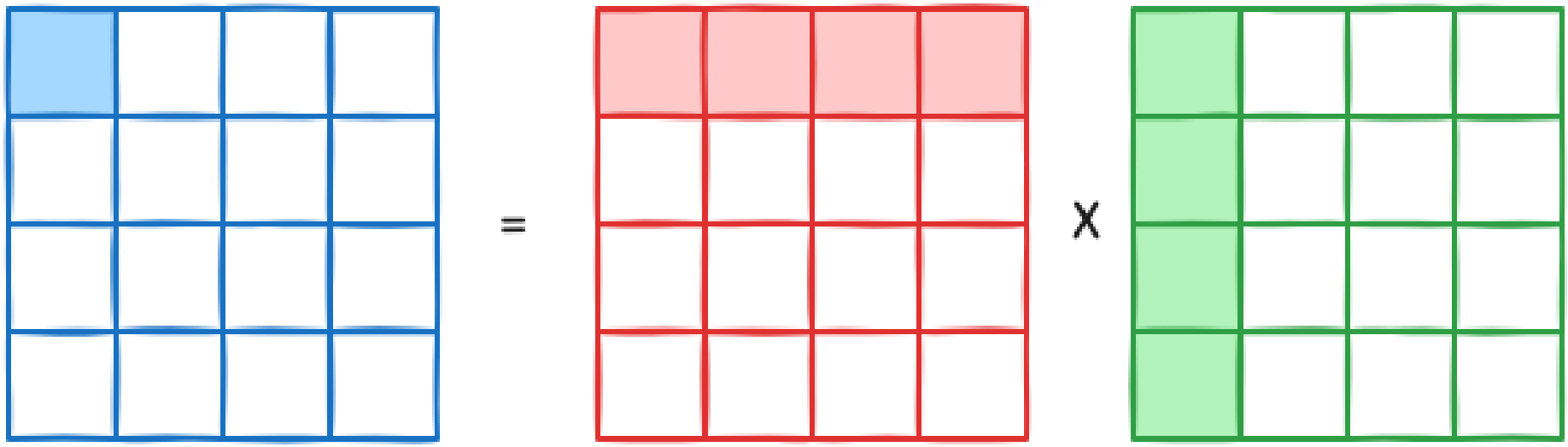


Each element of the output is independent. The results of C_{11} does not effect the results of C_{12}, or any other element.

Remark

If matrix multiplication is a **parallel** task, and GPU's were designed for the **parallel** task of rendering graphics, can we repurpose GPU's to do matrix multiplication?

Matrix Multiplication is Parallel



Each element of the output is independent. The results of C_{11} does not effect the results of C_{12}, or any other element.

Remark

If matrix multiplication is a **parallel** task, and GPU's were designed for the **parallel** task of rendering graphics, can we repurpose GPU's to do matrix multiplication? (Yes.)

Takeaway

- CPUs can't efficiently perform matrix multiplication at scale.
- Graphics rendering is a parallel task, and so is matrix multiplication. Thus, GPUs can be good at matrix multiplication.
- GEMM is a generalization of of matrix multiplication, and is the operation we will be discussing.

Whats next

We have established that GPUs *can* do matrix multiplication, but how do we actually get there?